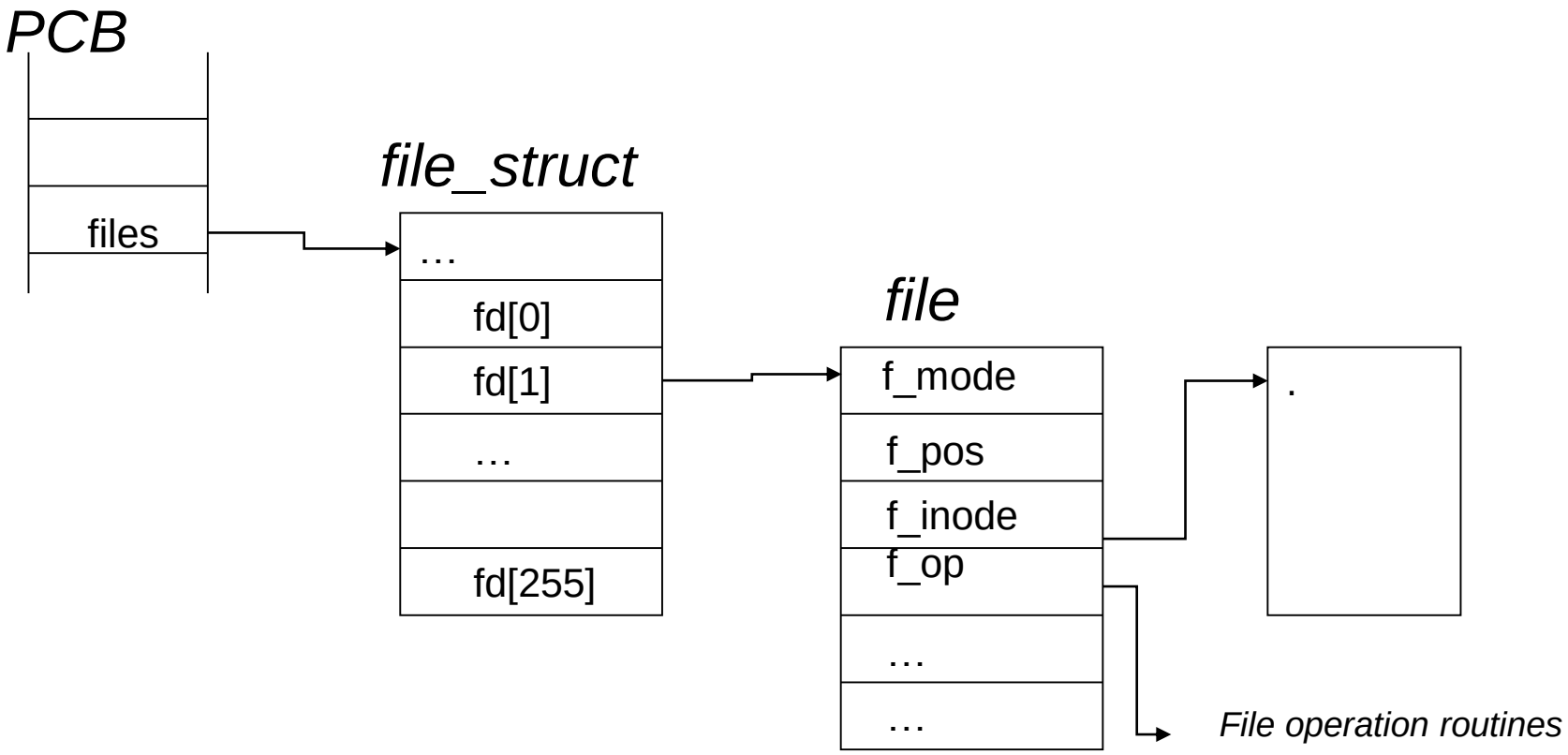
National University of Computer & Emerging Sciences

Lecture 12

IPC Unix Case Study - Named Pipes

File Descriptors

- The PCB of each process contains a pointer to a *file_struct, a kernel-resident array data structure containing the details of open files*



File Descriptors

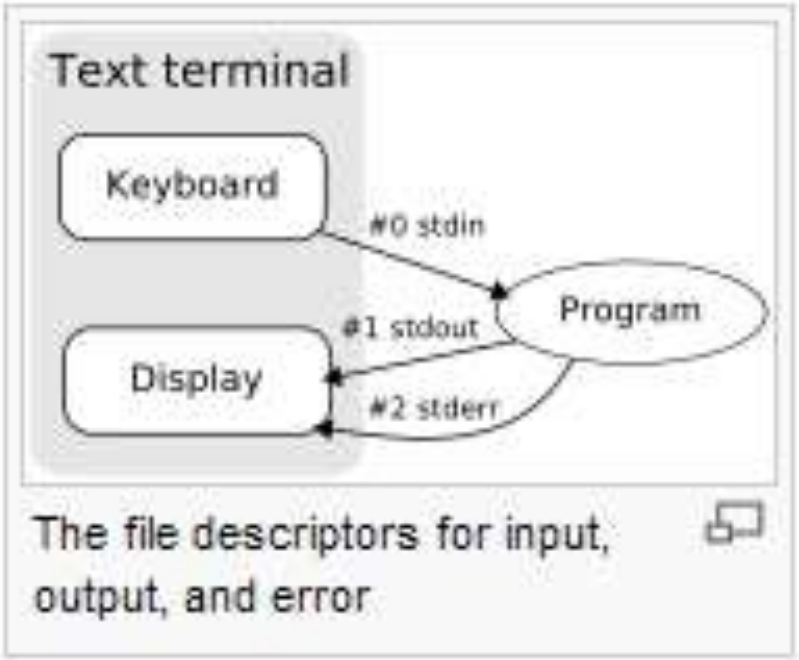
- The *files_struct* contains pointers to file data structures
- Each one describes a file being used by this process.
- *f_mode*: describes file mode, read only, read and write or write only.
- *f_pos*: holds the position in the file where the next read or write operation will occur.
- *f_inode*: points at the actual file

File Descriptors

- Every time a file is opened, one of the free file pointers in the *files_struct* is used to point to the new file structure.
- Linux processes expect three file descriptors to be open when they start.
- These are known as *standard input*, *standard output* and *standard error*

File Descriptors

- The program treat them all as files.
- These three are usually inherited from the creating parent process.
- All accesses to files are via standard system calls which pass or return file descriptors.
- *standard input*, *standard output* and *standard error* have file descriptors 0, 1 and 2.



The file descriptors for input, output, and error

File Descriptors

- char buffer[10];
- Read from standard input (by default it is keyboard)
  - read(0,buffer,5);
- Write to standard output (by default is is monitor))
  - write(1,buffer,5);
- By changing the file descriptors we can write to files
- fread/fwrite etc are wrappers around the above read/write functions

The Unix fact

- When Unix programs do any sort of I/O, they do it by reading or writing to a file descriptor
- A file descriptor is simply an integer associated with an open file

The Unix fact

- A file in Unix can be
  - A network connection
  - A FIFO queue
  - A pipe
  - A terminal
  - A real on-the-disk file
  - Or just about anything else

EVERYTHING IN UNIX IS A FILE!

Reading/Writing from/to a Pipe

- `int read(int filedescriptor, char *buffer, int bytetoread);`

- `int write(int filedescriptor,char *buffer,int bytetowrite);`

NamedPipes

- A FIFO is similar to a pipe. A FIFO (First In First Out) is a one-way flow of data. FIFOs have a name, so unrelated processes can share the FIFO. FIFO is a named pipe. This is the main difference between pipes and FIFOs.

mkfifo

int mkfifo(const char *pathname, mode_t mode);

- **pathname – a UNIX pathname (path and filename).**
- **The name of the FIFO mode – the file permission bits.**
  - **0: No special permissions are set.**
  - **6: For the owner (read and write permissions: rw-).**
  - **6: For the group (read and write permissions: rw-).**
  - **6: For others (read and write permissions: rw-).**

- **Thus, 0666 means that everyone (owner, group, and others) has read and write permissions, but no one has execute permission.**

Makefifo()

Each group has three permission types:

- **Read** (`r`): Permission to view the file's contents.
- **Write** (`w`): Permission to modify the file.
- **Execute** (`x`): Permission to execute the file (for programs/scripts) or enter the directory (for directories).

Each permission type is represented by a number:

- `r` (read) = 4
- `w` (write) = 2
- `x` (execute) = 1
- No permission = 0

mkfifo

- 1- After a FIFO is created, it can be opened for read or write.
- 2) Normally, opening a FIFO for read or write, it blocks until another process opens it for write or read.
- 3) A read gets as much data as it requests or as much data as the FIFO has, whichever is less.
- 4) A write to a FIFO is atomic, as long as the write does not exceed the capacity of the FIFO. The capacity is at least 4k

FIFO file

- A FIFO (named pipe) file created using mkfifo() appears in the filesystem,
- it does not store data on disk like a regular file.
- Instead, the data flows through kernel memory (typically in a buffer in RAM) when processes read from and write to the FIFO.

Named Pipe Example

Process 1

```
int main() {
   int fd;
   char * myfifo = "/tmp/myfifo";
   /* create the FIFO(named pipe)*/
   mkfifo(myfifo, 0666);

   /* write "Hi" to the FIFO */
   fd = open(myfifo, O_WRONLY);
   write(fd, "Hi", sizeof("Hi"));
   close(fd);

   /* remove the FIFO */
   unlink(myfifo);

   return 0;
}
```

Process 2

```
#define MAX_BUF 1024

int main() {
   int fd;
   char * myfifo = "/tmp/myfifo";
   char buf[MAX_BUF];

   /* open, read, and display the
      message from the FIFO */
   fd = open(myfifo, O_RDONLY);
   read(fd, buf, MAX_BUF);
   printf("Received: %s\n", buf);
   close(fd);

   return 0;
}
```

Redirection in C Programs

- Processes do not read from files, they read from file descriptors
- Close / Open Method:

```
close(0);                       /* file descriptor 0 – i.e. stdin */
open(filename, O_RDONLY);       /* redefines lowest file descriptor */
```

- Open / Close / Dup / Close / Method:

```
fd=open(filename, O_RDONLY);   /* open file to read - fd */
close(0);                      /* close file descriptor 0 – i.e.
                                  stdin */
newfd=dup(fd);                 /* make "clone" of fd and
                                  uses lowest next available fd –
                                  I.e. 0 that was closed */
close(fd);                     /* close original fd associated
                                  with file. Now, stdin is
                                  associated with file…   */
```

dup2 is similar to dup, but dup2 will automatically `close(0)`.
For example, `newfd=dup2(fd,0);`

dup

- duplicate an open file descriptor

```
fd_new = dup(int fd);
```

dup() takes the old file descriptor as the input parameter.

On success, it returns the lowest-numbered unused file descriptor as the new file descriptor.

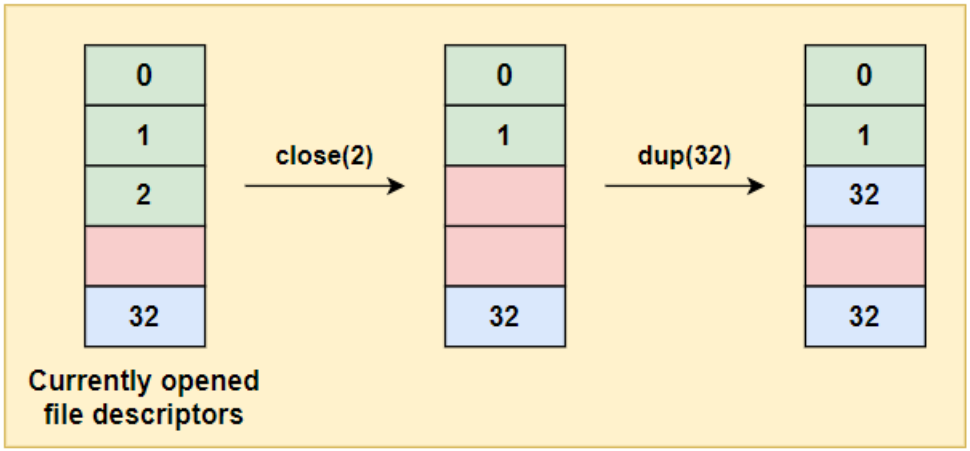
dup

- The dup() function, provided by the unistd.h library, is used in C programs to allow processes to duplicate existing file descriptors.
- The dup() function replaces the lowest-level unused file descriptor to place the copied file descriptor.

dup

- As for open(), the LOWEST unused file descriptor should be returned.
- Upon successful completion a non-negative integer, namely the file descriptor, shall be returned;
- otherwise, -1 shall be returned to indicate the error

dup


0
1
2
32
close(2)
0
1
32
dup(32)
0
1
32
32
Currently opened
file descriptors

dup

```
#include <stdio.h>
#include <unistd.h>
int main() {
  int file = open("output.txt", O_WRONLY | O_CREAT , 0644);
  if (file < 0) {
        perror("open");
        return 1;
   }
  int copy_fd = dup(file); // Creates a duplicate of file descriptor
  write(copy_fd, "Hello, dup!\n", 12); // Writing using the duplicate fd
  close(file);
  close(copy_fd);
  return 0;
}
```

Dup2

The dup2() system call is similar to dup() but the basic difference between them is that instead of using the lowest-numbered unused file descriptor, it uses the descriptor number specified by the user.

Syntax:

```
int dup2(int oldfd, int newfd);
oldfd: old file descriptor
newfd new file descriptor which is used by dup2() to create a copy.
```

Important points:

- Include the header file unistd.h for using dup() and dup2() system call.
- If the descriptor newfd was previously open, it is silently closed before being reused.
- If oldfd is not a valid file descriptor, then the call fails, and newfd is not closed.
- If oldfd is a valid file descriptor, and newfd has the same value as oldfd, then dup2() does nothing, and returns newfd.

Dup2 Example

```c
// CPP program to illustrate dup2()
#include<stdlib.h>
#include<unistd.h>
#include<stdio.h>
#include<fcntl.h>

int main()
{
    int file_desc = open("tricky.txt",O_WRONLY | O_APPEND);

    // here the newfd is the file descriptor of stdout (i.e. 1)
    dup2(file_desc, 1) ;

    // All the printf statements will be written in the file
    // "tricky.txt"
    printf("I will be printed in the file tricky.txt\n");

return 0;
}
```

Dup2

- *DUP2 IS USEFUL*

TO REDIRECT

- *STANDARD INPUT/OUTPUT*
- *(STDIN, STDOUT, STDERR).*

Redirecting stdout to a file:

```
int file = open("log.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(file, 1); // Now stdout goes to log.txt
printf("This will be written to log.txt\n");
```

Redirecting stdin from file

```
int file = open("input.txt", O_RDONLY);
dup2(file, 0); // Now stdin reads from input.txt
char buffer[100];
scanf("%s", buffer); //Reads from input.txt instead of keyboard
```

Redirecting stderr to a file (error logging):

```
int file = open("errors.log", O_WRONLY | O_CREAT | O_APPEND,
0644);
dup2(file, 2); // stderr redirected to errors.log
fprintf(stderr, "This error message goes to errors.log\n");
```